MuleSoft Vibes Demo

Created by: Tim Dai (MuleSoft Success Architect)
Created on: 2026-03-20

Demo Recording

https://www.youtube.com/watch?v=zHWYxvBYGXs

Demo Script

Goal

1. End-to-end API development lifecycle:

   Design API specification → Apply governance policies → Publish → Implement → MUnit testing → Deploy → Manage APIs → Monitor diagnostics → Generate insights

2. Driven entirely by prompt-based workflows, with no manually written code required.

Create API spec project

1. Open MuleSoft Vibes, use the following prompt (mcp server tool: create_api_spec_project, open_project):

```
None
Create a RAML MuleSoft API spec project:
- project name: demo-salesforce-accounts-api
- project path: /Users/liang.dai/Documents/01-dev/code-workspace

After the project is create, open it in the current VS Code workspace.
```

2. Vibes will ask a list of questions to create the project:
   - API Type: RAML 1.0
3. Click the “Approve” button to let Vibes generate the project. The project is created and opened into the current VSCode workspace.
4. Switch to MuleSoft Vibes, continue the current task chat. Provide below prompt to create the API spec (mcp server tool: generate_api_spec):

None

```
For the project
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api

Create a **RAML 1.0** API specification (single `.raml` file) for a
Salesforce-backed Accounts API with the following requirements:

### 1) Base

* Title: `Salesforce Accounts API`
* Version: `v1`
* Base URI example: `https://api.example.com/v1`
* Media type: `application/json`

### 2) Resource

Define a single resource:

* `/accounts`

### 3) GET behavior

Support **both** of these URL patterns:

* `GET /accounts`

  * Returns **all Accounts**.
  * Response `200` with a JSON array of account objects.
* `GET /accounts/{AccountNumber}`

  * Returns **only the account** matching `AccountNumber`.
  * Define `AccountNumber` as a **URI parameter** (string) with an example like
`DEMO123`.
  * If the account does not exist, return:

    * `404` with a JSON error body (include fields like `errorCode`, `message`).

### 4) PATCH behavior

Support:

* `PATCH /accounts/{AccountNumber}`

  * `AccountNumber` is required in the URL.
  * Request body: JSON object representing a **partial update** (fields + values
to update).
```

```
    * Keep it flexible (e.g., allow optional properties like `Name`, `Phone`,
`Website`, etc.), but make it clear it’s a partial update.
  * Responses:

    * `204 No Content` when update succeeds (no response body).
    * `404` with the same JSON error structure when the account does not exist.

### 5) Data types / examples

Include:

* A reusable `Account` type (object) with typical Salesforce fields (at minimum:
`AccountNumber`, `Name`; add a few realistic optional fields).
* A reusable `Error` type for the 404 response.
* Example payloads for:

  * `GET /accounts` response (array)
  * `GET /accounts/{AccountNumber}` response (single object)
  * `PATCH /accounts/{AccountNumber}` request (partial update object)
  * `404` error response

### 6) Output quality

* Use correct **RAML 1.0 syntax**.
* Prefer reusable `types`, and reference them in request/response bodies.
* Clearly document behavior in `description` fields (especially the “all accounts
vs single account” GET behavior and the PATCH success code `204`).

### 7) exchange.json

* Ensure the exchange.json file contains all mandatory metadata fields required
for publishing to Anypoint Exchange. A valid example structure is shown below:
{
  "main": "xxx.raml",
  "name": "xxx",
  "groupId": "572fe2f8-14c7-48e9-8ab3-2473e2fddccf",
  "assetId": "xxx",
  "version": "1.0.0",
  "classifier": "raml",
  "tags": [],
  "descriptorVersion": "1.0.0",
  "organizationId": "572fe2f8-14c7-48e9-8ab3-2473e2fddccf",
  "apiVersion": "1.0.0",
  "originalFormatVersion": "1.0"
```

```
}

### 8) Return only the RAML content.
```

5. Once the spec is created, show customers the API Console to run mocking service.

API Governance

1. Next, we will run API governance for the created API spec.
2. Continue the same task chat. Use below prompt to search the available API governance rulesets from Anypoint Exchange (mcp server tool: search_asset):

```
None
List the available API governance rulesets from Anypoint Exchange
```

3. Continue the same task chat. Use below prompt to add a ruleset (mcp server tool: add_governance_ruleset):

```
None
Add the API governance ruleset "Anypoint Best Practices" to the project:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api
```

4. Switching to VSCode File Explorer, we can see the “Anypoint Best Practices” ruleset is added into the project’s “Additional Rulesets”.
5. We can run validation either from Add Governance Rulesets > Additional Rulesets: Anypoint Best Practices or use below prompt (mcp server tool: validate_api_against_ruleset):

```
None
Validate the API specification in the project against governance ruleset "Anypiont
Best Practices"
```

6. For demo purpose, we will only fix following violations by using the prompt :

```
None
Fix the following violations:
- Operation Identifiers - All operations (GET, PATCH) must have `displayName`
properties
- Content-Type Headers - All responses (200, 404) must include explicit
`Content-Type` headers
```

Publish API to Exchange

1. Now we have the API spec ready, next, let’s publish it to Exchange. Before publishing the API, ***make sure to remove the added governance ruleset first***. Otherwise, it will

fail. Also, ***make sure to start a new task window***. Otherwise, it seems to memorise the previous validation errors.

2. Use below prompt (mcp server tool: create_and_manage_assets, validate_project):

```
None
Publish the API "Salesforce Accounts API" to Anypoint Exchange
Project path:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api
```

3. Go to Anypoint Exchange to review the published API. Next, we can talk about the feature - Generating API Documentation With Einstein Generative AI
4. Next, we will implement a project based on the API spec
5. Start a new task chat, use the below prompt to confirm the API asset is available in Exchange (mcp server tool: search_asset):

```
None
Search the API asset "Salesforce Accounts API" from Anypoint Exchange
```

API Implementation

1. Use below prompt to create an implementation project (mcp server tool: implement_api_spec, open_project):

```
None
Implement a MuleSoft application from the API spec: demo-salesforce-accounts-api
- Project Name: demo-salesforce-accounts-api-implementation
- Project Location: /Users/liang.dai/Documents/01-dev/code-workspace
- runtimeVersion: 4.11.0
- jdkVersion: 17
- For the time being, disregard the Salesforce Connector configuration and use the
sample mock data defined in the API specification instead.

After the project is create, open it in the current VS Code workspace.
```

2. Run the mule application locally (mcp server tool: run_local_mule_application):

```
None
Run the application:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
Use this Java Home:
/Users/liang.dai/AnypointCodeBuilder/java/jdk-17.0.13+11/Contents/Home/
```

3. The above prompt will pop up a MacOS terminal to start the app. It might fail due to: The Mule runtime is missing the required domains directory. If this happens, just copy the entire error message from the terminal to Vibes, and ask Vibes to fix it.

4. If the app starts successfully, Agentforce will call one of its endpoints to validate, then ask you to open the console in your browser:

```
None
curl -X GET "http://localhost:8081/api/accounts" -H "Accept: application/json"

open http://localhost:8081/console/
```

5. Close the terminal to stop the application.
6. Open the project:

```
None
Open the project
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
```

7. Before moving on, let's make a copy of the project in case we need to start again.
8. Update get:/accounts flow to use MuleSoft Salesforce Connector (mcp server tool: generate_mule_flow):
    a. Agentforce will update pom.xml, create global.xml, and update existing flow
    b. Then start building the updated application

```
None
Update the Mule flow get:/accounts in the
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation project to integrate with Salesforce using the latest MuleSoft
Salesforce Connector. Please don't use any Salesforce Connector versions which are
older than 11.0.0


Functional Requirements
1. Replace the existing implementation of get:/accounts with a Salesforce
Connector operation.
2. Use the Salesforce Connector (latest version available).
3. Use the Query All operation.
4. Execute the following SOQL query to retrieve the first 50 accounts:

SELECT FIELDS(All) FROM Account ORDER BY Name LIMIT 50

5. The flow should return the retrieved accounts as the HTTP response payload in
JSON format.

Salesforce Connection Configuration
```

```
Configure the connector using:
1. Connection Type: oauth-client-credentials-connection
2. Token URL:
https://orgfarm-6769b836dc-dev-ed.develop.my.salesforce.com/services/oauth2/token
3. Client ID: use placeholder ${secure::salesforce.clientId}
4. Client Secret: use placeholder ${secure::salesforce.clientSecret}

Security Requirements (Mandatory)

Do NOT hardcode the client ID or client secret in the Mule configuration.
Instead:
1. Externalize them using secure configuration:
       1.1 Create a new secure-config.yaml property file
       1.2 Store salesforce.clientId and salesforce.clientSecret in the file. For
instance:
       salesforce:
         clientId: "![encrypted client id]"
         clientSecret: "![encrypted client secret]"
2. Reference them in the connector configuration using ${} placeholders. For
instance: ${secure::salesforce.clientId}, ${secure::salesforce.clientSecret}
3. Clearly indicate where encrypted values should be placed.
4. Add Secure Configuration Properties dependency to `pom.xml`
5. Create encrypted properties file using MuleSoft Secure Properties Tool
6. Configure the secure properties module in `global.xml`
       6.1 encryption key: MySecretKey1234!
       6.2 Entryption method: Blowfish
       6.3 mode: CBC
7. The encrypted properties:
       7.1 for clientId:
uVWXionBMSA0sDINLkfX1qsRyOPU4/wmwvIUMTo+PFOPCOQiJIWuYm+bQrl+sDDtvg5xH/t//BOVVtEVkN
xdpjtcXYYColAX1pctYJ4t0px7KJSnTk6G/w==
       7.2 for clientSecret:
qKGZpJ1nyk8f83rNS4fowLu7kYXmJJgkE+5xj+qqVJ9T7aSS9G7ejQcbqVqF7wqjlUenP+Iq32vQqGcAyj
c9HZLvLuPKJZ9d
```

9. Above encrypted properties, is using [MuleSoft Security Properties Tool](). We don't need to run this step in the demo.

```
None
For client id:
===
java -cp secure-properties-tool.jar com.mulesoft.tools.SecurePropertiesTool \
  string encrypt Blowfish CBC MySecretKey1234! \
```

```
"3MVG9dAEux2v1sLtT6nYYWDPTZgezwssP.FofQYkhBwlg4ppxXxbH3Ms6pCdYSlveZkL5D7iIAcRii7oj
jphn"

For client secret:
===
java -cp secure-properties-tool.jar com.mulesoft.tools.SecurePropertiesTool \
  string encrypt Blowfish CBC MySecretKey1234! \
  "FCCB27E59059370615A9BE6BBD1AA55CE2B4381AF8D5E69EB6C6D8D7D6D91F8A"
```

10. Start a new task window
11. Run the update application and test get:/accounts endpoint. It will open an external terminal. To avoid confusion, close all existing external terminals. If there are errors from the terminal, copy and paste the errors to Agentforce.

```
None
Run the application:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
Use this Java Home:
/Users/liang.dai/AnypointCodeBuilder/java/jdk-17.0.13+11/Contents/Home/

Test the get:/accounts endpoint
```

12. Start a new task window
13. Implement get:/accounts/{AccountNumber} flow:

```
None
Update the Mule flow get:/accounts/{AccountNumber} in the
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation project.

Functional Requirements
1. Replace the existing implementation of get:/accounts with a Salesforce
Connector operation.
2. Reuse the existing configure Salesforce Connector.
3. Use the Query All operation.
4. Construct a SOQL query to retrieve a single Account record using the
AccountNumber value provided in the request URL. Extract the {AccountNumber} value
from the URL path parameter. Construct the following SOQL query dynamically:
```

```
SELECT FIELDS(All) FROM ACCOUNT WHERE AccountNumber = '{AccountNumber}' LIMIT 1

5. The flow should return the retrieved accounts as the HTTP response payload in
JSON format.
6. If the account doesn't exist, return 404 error.
```

14. Run the update application and test get:/accounts/{AccountNumber} endpoint. It will open an external terminal. To avoid confusion, close all existing external terminals. If there are errors from the terminal, copy and paste the errors to Agentforce.

```
None
Run the application:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
Use this Java Home:
/Users/liang.dai/AnypointCodeBuilder/java/jdk-17.0.13+11/Contents/Home/

Test the get:/accounts/{AnypointNumber} endpoint. For testing purpose, use
AccountNumber = DEMO123
```

15. Start a new task window
16. Implement patch:/accounts/{AccountNumber} flow:

```
None
Update the Mule flow patch:/accounts/{AccountNumber} in the
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation project.

Functional Requirements
1. Replace the existing implementation of patch:/accounts{AccountNumber}
2. Do not create a new Salesforce connector configuration. Reuse the existing
configured Salesforce Connector in the project.
3. The incoming request body contains the fields that must be updated. Example
payload:
{
    "BillingStreet" : "1 Lovely Street",
    "Phone": "88889999"
}
Immediately store the full request body in a variable named: v_payload.
4. se the Salesforce Query All operation to retrieve the Account record.
```

```
5. Construct a SOQL query to retrieve a single Account record using the
AccountNumber value provided in the request URL. Extract the {AccountNumber} value
from the URL path parameter. Construct the following SOQL query dynamically:


SELECT FIELDS(All) FROM ACCOUNT WHERE AccountNumber = '{AccountNumber}' LIMIT 1

6. From the result of the Query All operation:
       - If a record is returned: Extract the Id field from the first record.
       - If no record is returned: Immediately return HTTP 404 Not Found, and Stop
further processing.

7. Create a new payload by merging:
       - The extracted Id
       - The original request body stored in v_payload
The merged payload is an array. Expected structure:
[
       {
         "Id": "001XXXXXXXXXXXXXXX",
         "BillingStreet": "1 Lovely Street",
         "Phone": "88889999"
       }
]

8. Perform the Update Operation - Use the Salesforce Connector:
       - Operation: Update
       - Object Type: Account
       - Payload: The combined object created in Step 7
This operation should update only the fields provided in the original request
body.
```

17. Use the below prompt to run the updated application and test the patch method. Pay attention to the popped up terminal window, and wait for the app uprunning before run the test command

```
None
Run the application:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
Use this Java Home:
/Users/liang.dai/AnypointCodeBuilder/java/jdk-17.0.13+11/Contents/Home/
```

```
Test the patch:/accounts/{AnypointNumber} endpoint. For testing purpose, use
AccountNumber = DEMO123, request payload body is:
{
    "BillingStreet" : "1 Lovely Street",
    "Phone": "88889999"
}
```

18. Use AI for Data Weave transformation. Open the project explorer and locate the get:/accounts flow. Click the Transformation Message component, then click “Transformation Builder with Map with AI”. Switch to “Map and Transform with AI”. Use below input and output sample. Apart from the smaller number of fields in the output payload, also pay attention to the format of “CreatedDate” is changed too.

Input Sample:


```
None
[
    {
        "ShippingLatitude": null,
        "LastModifiedDate": "2025-04-15T21:07:53.000Z",
        "ShippingStateCode": null,
        "BillingCity": "Burlington",
        "JigsawCompanyId": null,
        "SLA__c": "Silver",
        "type": "Account",
        "NaicsCode": null,
        "Name": "Burlington Textiles Corp of America",
        "Industry": "Apparel",
        "CreatedById": "005gL000001oyjZQAQ",
        "OperatingHoursId": null,
        "TickerSymbol": "BTXT",
        "BillingGeocodeAccuracy": null,
        "AccountSource": null,
        "BillingPostalCode": "27215",
        "DandbCompanyId": null,
        "PhotoUrl": "/services/images/photo/001gL000004A8WhQAK",
        "MasterRecordId": null,
        "IsDeleted": "false",
        "LastViewedDate": "2026-02-18T02:14:55.000Z",
        "ShippingGeocodeAccuracy": null,
        "DunsNumber": null,
        "Sic": "546732",
        "ShippingStreet": null,
```

```
        "CleanStatus": "Pending",
        "BillingStateCode": "NC",
        "ShippingPostalCode": null,
        "YearStarted": null,
        "CreatedDate": "2025-04-15T21:07:53.000Z",
        "CustomerPriority__c": null,
        "ShippingState": null,
        "Id": "001gL000004A8WhQAK",
        "SicDesc": null,
        "BillingState": "NC",
        "AnnualRevenue": "3.5E8",
        "Jigsaw": null,
        "Active__c": null,
        "Site": null,
        "Ownership": "Public",
        "Description": null,
        "Rating": "Warm",
        "BillingCountryCode": "US",
        "Website": "www.burlington.com",
        "LastReferencedDate": "2026-02-18T02:14:55.000Z",
        "BillingLatitude": null,
        "NumberOfEmployees": "9000",
        "BillingAddress": {
            "country": "USA",
            "city": "Burlington",
            "countryCode": "US",
            "street": "525 S.  Lexington Ave",
            "latitude": null,
            "postalCode": "27215",
            "geocodeAccuracy": null,
            "stateCode": "NC",
            "state": "NC",
            "longitude": null
        },
        "OwnerId": "005gL000001oyjZQAQ",
        "SLASerialNumber__c": "5367",
        "BillingLongitude": null,
        "Tradestyle": null,
        "Phone": "(336) 222-7000",
        "NumberofLocations__c": "6.0",
        "UpsellOpportunity__c": "Maybe",
        "ShippingCountry": null,
        "ShippingCity": null,
        "ParentId": null,
        "NaicsDesc": null,
```

```
        "ShippingCountryCode": null,
        "AccountNumber": "CD656092",
        "SystemModstamp": "2025-04-15T21:07:53.000Z",
        "Type": "Customer - Direct",
        "BillingCountry": "USA",
        "SLAExpirationDate__c": "2025-05-09",
        "BillingStreet": "525 S. Lexington Ave",
        "ShippingAddress": null,
        "LastActivityDate": null,
        "Fax": "(336) 222-8000",
        "LastModifiedById": "005gL000001oyjZQAQ",
        "ShippingLongitude": null
    },
    {
        "ShippingLatitude": null,
        "LastModifiedDate": "2026-02-23T00:37:21.000Z",
        "ShippingStateCode": null,
        "BillingCity": "Melbourne",
        "JigsawCompanyId": null,
        "SLA__c": null,
        "type": "Account",
        "NaicsCode": null,
        "Name": "Demo Company",
        "Industry": null,
        "CreatedById": "005gL0000027tWTQAY",
        "OperatingHoursId": null,
        "TickerSymbol": null,
        "BillingGeocodeAccuracy": null,
        "AccountSource": null,
        "BillingPostalCode": "3000",
        "DandbCompanyId": null,
        "PhotoUrl": "/services/images/photo/001gL00000ghfW2QAI",
        "MasterRecordId": null,
        "IsDeleted": "false",
        "LastViewedDate": "2026-02-23T00:38:12.000Z",
        "ShippingGeocodeAccuracy": null,
        "DunsNumber": null,
        "Sic": null,
        "ShippingStreet": null,
        "CleanStatus": "Pending",
        "BillingStateCode": "VIC",
        "ShippingPostalCode": null,
        "YearStarted": null,
        "CreatedDate": "2026-02-18T02:16:35.000Z",
        "CustomerPriority__c": null,
```

```
        "ShippingState": null,
        "Id": "001gL00000ghfW2QAI",
        "SicDesc": null,
        "BillingState": "Victoria",
        "AnnualRevenue": null,
        "Jigsaw": null,
        "Active__c": null,
        "Site": null,
        "Ownership": null,
        "Description": null,
        "Rating": null,
        "BillingCountryCode": "AU",
        "Website": null,
        "LastReferencedDate": "2026-02-23T00:38:12.000Z",
        "BillingLatitude": null,
        "NumberOfEmployees": null,
        "BillingAddress": {
            "country": "Australia",
            "city": "Melbourne",
            "countryCode": "AU",
            "street": "5 Lovely Street",
            "latitude": null,
            "postalCode": "3000",
            "geocodeAccuracy": null,
            "stateCode": "VIC",
            "state": "Victoria",
            "longitude": null
        },
        "OwnerId": "005gL0000027tWTQAY",
        "SLASerialNumber__c": null,
        "BillingLongitude": null,
        "Tradestyle": null,
        "Phone": "88889999",
        "NumberofLocations__c": null,
        "UpsellOpportunity__c": null,
        "ShippingCountry": null,
        "ShippingCity": null,
        "ParentId": null,
        "NaicsDesc": null,
        "ShippingCountryCode": null,
        "AccountNumber": "DEMO123",
        "SystemModstamp": "2026-02-23T00:37:21.000Z",
        "Type": null,
        "BillingCountry": "Australia",
        "SLAExpirationDate__c": null,
```

```
        "BillingStreet": "5 Lovely Street",
        "ShippingAddress": null,
        "LastActivityDate": null,
        "Fax": null,
        "LastModifiedById": "005gL0000027tWTQAY",
        "ShippingLongitude": null
    }
]
```


Output Sample:


```
None
[
    {
        "AccountNumber": "CD656092",
        "Industry": "Apparel",
        "CreatedDate": "2025-04-15",
        "Id": "001gL000004A8WhQAK",
        "AnnualRevenue": "3.5E8",
        "Ownership": "Public",
        "Website": "www.burlington.com",
        "NumberOfEmployees": "9000",
        "BillingAddress": {
            "country": "USA",
            "city": "Burlington",
            "countryCode": "US",
            "street": "525 S. Lexington Ave",
            "latitude": null,
            "postalCode": "27215",
            "geocodeAccuracy": null,
            "stateCode": "NC",
            "state": "NC",
            "longitude": null
        }
    },
    {
        "AccountNumber": "DEMO123",
        "Industry": null,
         "CreatedDate": "2026-02-18",
        "Id": "001gL00000ghfW2QAI",
        "AnnualRevenue": null,
        "Ownership": null,
        "Website": null,
```

```
        "NumberOfEmployees": null,
        "BillingAddress": {
            "country": "Australia",
            "city": "Melbourne",
            "countryCode": "AU",
            "street": "5 Lovely Street",
            "latitude": null,
            "postalCode": "3000",
            "geocodeAccuracy": null,
            "stateCode": "VIC",
            "state": "Victoria",
            "longitude": null
        }
    }
]
```

19. Click the “Transform” button. Click “View Suggested Script”. Click the “Insert” button.
20. Run the app again, then call get:/accounts via POSTMAN to verify the result.

Generate MUnit Test

1. Generate munit test with the following prompt (mcp server tool: generate_or_modify_munit):

```
None
For the project:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation, generate comprehensive MUnit test cases for the following Mule flows:

1. `get:/accounts`
2. `get:/accounts/{AccountNumber}`
3. `patch:/accounts/{AccountNumber}`

Requirements
1. Coverage
   1.1 Create test cases for both:
     - Successful execution scenarios
     - Failure or error scenarios (e.g., record not found, validation failure,
connector error)

2. Mocking
   2.1 Mock all external dependencies (e.g., Salesforce Connector operations).
```

```
   2.2 Provide realistic mock responses for each scenario.
   2.3 Ensure mocks align with the expected structure of Salesforce responses.

3. Test Data
   3.1 Generate appropriate and valid mock payloads for:
     - Request inputs
     - Connector responses
     - Error conditions
   3.2 Ensure test data reflects real-world Account records.

4. Assertions
   4.1 Include meaningful assertions to validate:
     - HTTP status codes
     - Response payload structure and values
     - Error handling behavior

5. Structure
   5.1 Follow MUnit best practices:
     - Proper use of `munit:behavior`
     - Clear test naming conventions
     - Isolation of test cases
```

2. Use below prompt to run munit test:

```
None
Run munit test for the project:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
```

3. During the MUnit execution, you may encounter test failures or observe low coverage metrics. Use the prompt below to iteratively refine and improve the test suite. Each execution of the prompt should address identified gaps and incrementally increase coverage. For demonstration purposes, achieving 100% coverage is not required.

```
None
Fix the errors or issues from the munit test and try to get the test coverage to
100%
```

Deploy the Application

1. Deploy the application to CH2.0 private space (mcp server tool: deploy_mule_application):

```
None
Deploy the application to CloudHub 2.0:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation

Requirements:
- app name: demo-salesforce-accounts-api-impl
- environment name: Sandbox
- runtime version: 4.11.2:3e-java17
- deployment target name: demo-ps
- replica size: 0.1 vCore
- number of replica: 1
```

2. Check the deployed application (mcp server tool: list_applications):

```
None
Retrieve the deployment details for the application
demo-salesforce-accounts-api-impl in the Sandbox environment.
```

3. From the returned details, find the public URL, run a couple of API calls via POSTMAN.
4. Create a new API instance (mcp server tool: create_and_manage_api_instances, list_api_instances, search_asset):

```
None
Create a new API instance for API asset: demo-salesforce-accounts-api
- asset type: HTTP
- runtime engine": mule4,
- mule proxy type": basicEndpoint
```

5. Go to Runtime Manager and add below properties. Without this update, the app deployment will not be successful. Make sure to protect the values.

```
None
anypoint.platform.client_id=xxx
anypoint.platform.client_secret=xxx
```

API Management

1. Use the API instance id to update the application's Auto Discovery ID in the global setting:

```
None
Update the Mule application demo-salesforce-accounts-api-implementation to
configure the API Autodiscovery ID in global.xml so that it matches the existing
API Instance ID for the managed API:
- API Instance Name: demo-salesforce-accounts-api
- Application Path:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
- If global.xml doesn't exist yet, create it.

Requirements
1. Add the api-platform-gw:api (API Autodiscovery) configuration in global.xml.
This is the only place to add the configuration. An sample configuration:
<api-gateway:autodiscovery apiId="YOUR_API_ID_HERE" flowRef="currentTimeFlow"
doc:name="Autodiscovery" />

xsi:schemaLocation="http://www.mulesoft.org/schema/mule/api-gateway
http://www.mulesoft.org/schema/mule/api-gateway/current/mule-api-gateway.xsd"

2. Set the apiId (Autodiscovery ID) to the correct API Instance ID associated with
demo-salesforce-accounts-api.
3. Ensure the configuration is valid and deployable.
4. Confirm that the updated Autodiscovery ID enables the application to
successfully link to the API instance in API Manager after deployment.

The outcome should ensure that, once deployed, the Mule application correctly
registers with and is governed by the intended API instance in API Manager.
```

2. Run the application to make sure it still works, this will also generate a new target with the updated code for the redeployment:

```
None
Run the application:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation
Use this Java Home:
/Users/liang.dai/AnypointCodeBuilder/java/jdk-17.0.13+11/Contents/Home/
```

3. Redeploy the application:

```
None
Go to Runtime Manager and upload the new .jar file to the application for
deployment:
/Users/liang.dai/Documents/01-dev/code-workspace/demo-salesforce-accounts-api-impl
ementation/target/demo-salesforce-accounts-api-implementation-1.0.0-mule-applicati
on.jar
```

4. In case the API instance still shows as “Unregistere”:

```
None
Wait for a minute
```

5. Apply rate limiting policy (mcp server tool: [manage_api_instance_policy]):

```
None
Apply a rate limiting policy to the API instance demo-salesforce-accounts-api with
a limit of 5 requests per 60 seconds.
```

Diagnostics Agent & Insights

1. Diagnostics Agent

```
None
Go to Runtime Manager > find the application > Diagnositice > Click Analyze &
Troubleshoot
```

2. Platform insights

```
None
Provide usage and operational insights for the application:
demo-salesforce-accounts-api-impl
```

Code Review

Prompt:

None

```
You are a senior software engineer performing a code review.
Project path: /Users/liang.dai/Documents/01-dev/code-workspace/test-mule-tool
Please review the code for:
- Code clarity and readability
- Maintainability and modularity
- Adherence to general programming best practices
- Opportunities for simplification or improvement
- Potential code smells or anti-patterns
- Sensitive data is handled securely (e.g., no hardcoded secrets)
- Proper exception handling and logging mechanisms are in place
- There are any performance bottlenecks
- External libraries or APIs used are correctly and safely
- The code is compliant with integration patterns or architectural guidelines
- Version constraints or compatibility are considered for dependencies
- If the file content is empty, don't include it in the response
- Severity: Major / Minor / Suggestion
- Clear recommendation or code snippet fix
- (If relevant) link to MuleSoft documentation page
```